



Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Relatório do Laboratório 03 - Redes Neurais Convolucionais

1 Breve Explicação em Alto Nível da Implementação

A rede neural implementada segue a arquitetura clássica LeNet-5, proposta originalmente para o reconhecimento de dígitos manuscritos no conjunto de dados MNIST. A rede recebe como entrada imagens em escala de cinza de tamanho 28×28 pixels e produz como saída uma pontuação para cada uma das 10 classes possíveis (dígitos de 0 a 9).

O modelo é composto por três camadas convolucionais e duas camadas totalmente conectadas, organizadas da seguinte forma:

- **Primeira camada convolucional:** Aplica 6 filtros de tamanho 5×5 sobre a imagem de entrada, extraindo características simples como bordas e contornos. Em seguida, uma camada de *pooling* reduz as dimensões espaciais pela metade, resumindo a informação local.
- **Segunda camada convolucional:** Aplica 16 filtros de tamanho 5×5 sobre o resultado anterior, combinando as características simples em padrões mais complexos. Novamente, uma camada de *pooling* reduz as dimensões.
- **Terceira camada convolucional:** Aplica 120 filtros de tamanho 5×5 , produzindo uma representação compacta e rica da imagem. Após essa camada, o resultado é convertido em um vetor unidimensional.
- **Camadas totalmente conectadas:** O vetor de 120 valores passa por duas camadas densas — a primeira reduz para 84 unidades e a segunda produz os 10 valores finais, um para cada classe.

Após cada camada convolucional e após a primeira camada densa, aplica-se a função de ativação tangente hiperbólica (*tanh*), que introduz não-linearidade e permite que a rede aprenda padrões mais complexos. A última camada não possui função de ativação, pois a função de perda utilizada no treinamento (entropia cruzada) já incorpora internamente a operação de *softmax*.

O *pooling* utilizado em todas as camadas é do tipo médio (*average pooling*), com janela 2×2 e passo 2, o que reduz cada dimensão espacial à metade.

2 Figuras Comprovando Funcionamento do Código

As figuras a seguir apresentam as curvas de treinamento da rede, exemplos de classificações corretas e exemplos de imagens classificadas incorretamente.

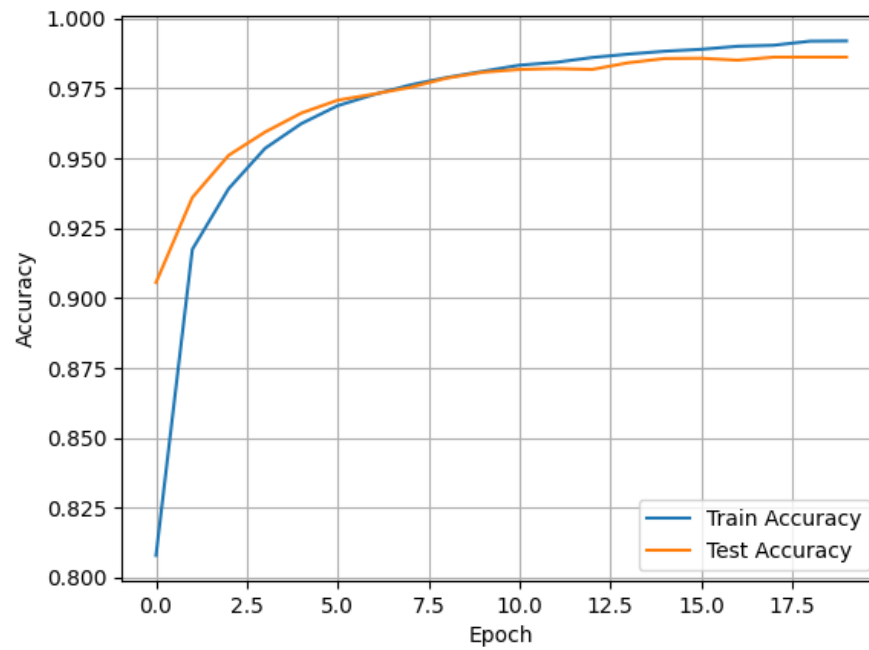


Fig. 1: Acurácia de treino e teste ao longo das épocas. A rede atinge rapidamente valores acima de 95% e estabiliza próximo a 99% no treino e 98% no teste.

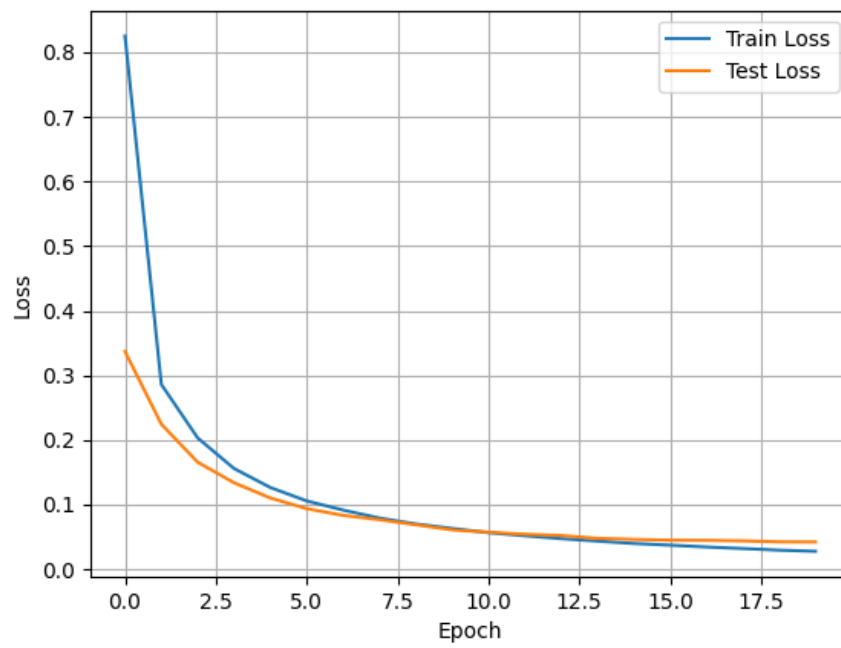
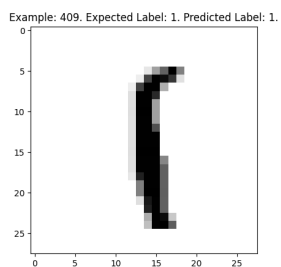
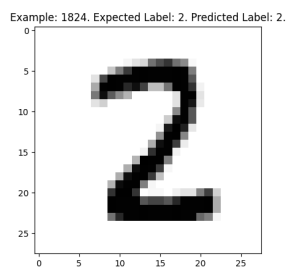


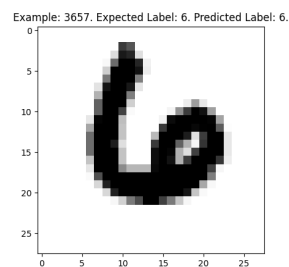
Fig. 2: Perda de treino e teste ao longo das épocas. A função de perda decresce de forma consistente e converge para valores próximos de zero.



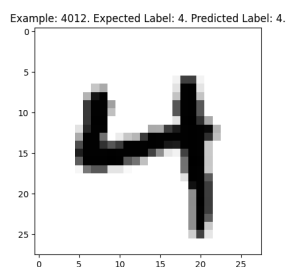
(a) Exemplo 409: esperado 1, predito 1.



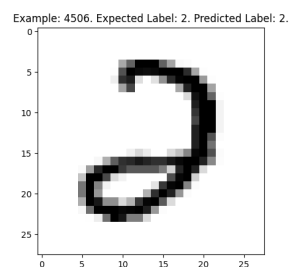
(b) Exemplo 1824: esperado 2, predito 2.



(c) Exemplo 3657: esperado 6, predito 6.

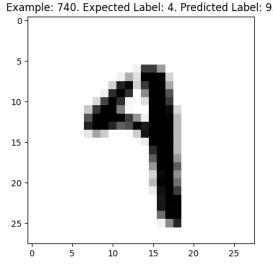


(d) Exemplo 4012: esperado 4, predito 4.

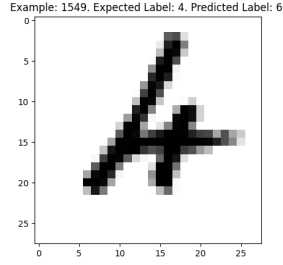


(e) Exemplo 4506: esperado 2, predito 2.

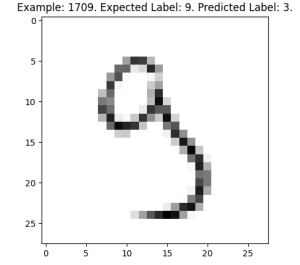
Fig. 3: Exemplos de imagens classificadas corretamente pelo modelo.



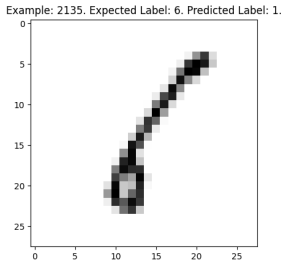
(a) Exemplo 740: esperado 4, predito 9.



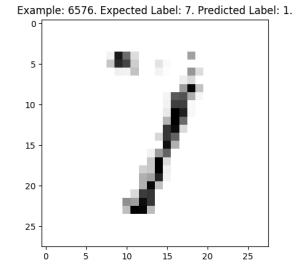
(b) Exemplo 1549: esperado 4, predito 6.



(c) Exemplo 1709: esperado 9, predito 3.



(d) Exemplo 2135: esperado 6, predito 1.



(e) Exemplo 6576: esperado 7, predito 1.

Fig. 4: Exemplos de imagens classificadas incorretamente pelo modelo.

3 Discussão dos Resultados

Curvas de treinamento. As Figs. 1 e 2 mostram que o modelo aprendeu de forma estável ao longo das 20 épocas de treinamento. A acurácia sobe rapidamente nas primeiras épocas — saindo de cerca de 81% no treino e 91% no teste já na primeira época — e se estabiliza próximo a 99% no conjunto de treino e 98% no conjunto de teste. A perda, por sua vez, cai de forma consistente e converge para valores próximos de zero. Um ponto interessante é que, nas primeiras épocas, a acurácia de teste supera a de treino, o que é comum quando se utiliza técnicas como *dropout* ou quando o conjunto de treino é percorrido em lotes ao longo da época, que é o caso desta rede, conforme mostra o código modelo. A partir de certo ponto, treino e teste se estabilizam próximos, sem sinal de *overfitting* relevante, o que indica que o modelo generaliza bem para dados não vistos.

Classificações corretas. A Fig. 3 apresenta cinco exemplos classificados corretamente. Nota-se que o modelo acerta mesmo em casos com escrita pouco uniforme, como o “1” do exemplo 409, que é fino e levemente inclinado. Isso demonstra que as camadas convolucionais conseguem capturar as características essenciais dos dígitos independentemente de pequenas variações de estilo.

Erros de classificação. A Fig. 4 mostra cinco exemplos em que o modelo falhou. Em geral, os erros ocorrem em imagens com escrita ambígua ou incomum:

- O exemplo 740 (esperado: 4, predito: 9) apresenta um “4” com traços que se assemelham visualmente ao “9”, pois a parte superior do dígito forma uma curva fechada.
- O exemplo 1549 (esperado: 4, predito: 6) mostra um “4” com estilo aberto e inclinado, cujo traço inferior pode sugerir um “6” ao modelo.
- O exemplo 1709 (esperado: 9, predito: 3) exibe um “9” com a cauda muito curvada, tornando-o visualmente próximo ao “3”.
- O exemplo 2135 (esperado: 6, predito: 1) apresenta um “6” escrito de forma muito alongada e estreita, com pouco fechamento na parte inferior, o que o torna parecido com um “1”.
- O exemplo 6576 (esperado: 7, predito: 1) mostra um “7” sem o traço horizontal central, tornando-o quase indistinguível de um “1”.

Em todos os casos, os erros são compreensíveis do ponto de vista visual: trata-se de dígitos escritos de forma atípica, que talvez não causariam dúvida em um leitor humano, mas, para uma rede neural que aprendeu a considerar uma matriz de pixels para aprender de qual dígito se trata, são variações que podem influenciar a classificação. Todavia, para um grande volume de dados, esses erros tendem a se tornar pouco relevantes. Isso reforça que o modelo atingiu um bom nível de desempenho e que os erros remanescentes estão concentrados em casos genuinamente difíceis do ponto de vista analítico de uma CNN.